



MYTHIC

YOLOPX Retraining Guide for Mythic AMP

May 2026

Document Version 1.0
SDK Version v26.05.0

1	Introduction	3
2	Hardware Requirements	3
3	Prerequisites	3
4	Obtaining the dataset, FP32 training and ONNX export	3
5	Model-zoo Configuration	3
5.1	Dataset Setup	4
6	Conversion and Retraining Steps	4
6.1	Convert the Floating Point model to Structural	4
6.2	Convert the Structural Model to Retraining	4
6.3	Retrain	5
6.3.1	Hyper-parameters	5
6.4	Optimize per-channel ADC ranges (optional)	5
6.5	Compute Accuracy Metrics	6
6.5.1	M2000	6
7	Convert to ACM Model	6
7.1	Generate Artifact	6
7.2	9. Firmware Compilation	7

MYTHIC

1 Introduction

Mythic[®] provides the YOLOPX panoptic driving perception model as part of its model-zoo. This guide provides instructions on how to train YOLOPX for Mythic's Analog Matrix Processor (AMP). The steps to train a model are:

1. Train the FP32 model. This is used as a starting point for Mythic's analog-aware training.
2. Convert the FP32 PyTorch model to ONNX. Mythic uses ONNX as a standard format to its training tools. This allows frameworks other than PyTorch to generate the initial FP32 model.
3. Convert the FP32 model for analog-aware retraining.
4. Perform analog-aware retraining.
5. Verify the accuracy of the analog-aware model using a software simulation of the Mythic AMP. This is essential in confirming that the model will provide good silicon accuracy.
6. Compile and deploy the mode to silicon to confirm final latency and accuracy.

2 Hardware Requirements

Model retraining is a compute-intensive process that requires a PC with sufficient performance to meet training demands.

Mythic recommends the following system specifications:

- Intel Core i7 CPU or better
- 64GB of RAM
- NVIDIA GPU with 48GB of VRAM
- Ubuntu 22.04 or newer

Lower performance systems may also work, but they will require tuning hyperparameters such as batch size to fit the model in memory.

For more information, please contact Mythic's Support Team.

3 Prerequisites

The following is required for analog-aware training:

1. A workstation with a GPU and CUDA installed. See the above for the recommended GPU configuration.
2. Mythic's SDK has been installed either natively or as Docker container.

4 Obtaining the dataset, FP32 training and ONNX export

Mythic uses a modified version of the YOLOPX repository, which is available at <https://github.com/mythic-external-collaboration/YOLOPX/tree/main>.

Refer to the README.md of the YOLOPX repository for the instructions on downloading the dataset, fp32 training, evaluation and export to ONNX. After the completion of these steps you should have a trained fp32 model exported to ONNX - by default `output.onnx`. These steps don't require model-zoo or any Mythic dependencies.

The subsequent steps are performed in model-zoo within the Mythic SDK.

5 Model-zoo Configuration

1. Set the environment variable `MYTHIC_SDK_ROOT` to point to the SDK directory. For example,

MYTHIC

```
export MYTHIC_SDK_ROOT=~/.mythic_sdk/v26.05.0/
```

2. Go to the model-zoo source code directory:

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo
```

3. Mythic's analog-aware training needs certain environment variables to be set depending on the task being trained. A script is provided to activate the `mythic-model-zoo` Python environment and the environment variables for the training task you want to work with.

- a. For 720p YOLOPX training:

```
./scripts/yolopx/yolopx-720p-m2000.env
```

The Mythic model-zoo scripts use Hydra to handle configuration files and command line parameters. The examples in the following sections show some basic ways of overriding configuration parameters. For advanced usage, please refer to the Hydra documentation.

5.1 Dataset Setup

Model-zoo training uses the same dataset as fp32 training in the YOLOPX repo. Set the environment variable `DATASET` to the location of the dataset.

```
export DATASET=your/dataset/directory/bdd100k/
```

6 Conversion and Retraining Steps

Each of the following steps generates an output file. Please confirm that a valid output has been generated before proceeding to the next step.

6.1 Convert the Floating Point model to Structural

A "structural" model is an intermediate model in the conversion process from floating point to retraining. This step requires the ONNX export output you have obtained earlier in the YOLOPX repository as described above. To obtain the structural model run:

```
python3 scripts/common/convert_model.py steps=to_structural fp_model=your/yolopx/export/output.onnx
```

Output: The converted model will be saved under `data/structural.onnx`

6.2 Convert the Structural Model to Retraining

Conversion from structural to retraining is done with the `to_training` step:

MYTHIC

```
python3 scripts/common/convert_model.py steps=to_training
```

Output: The retraining ONNX `data/mythic.onnx`

6.3 Retrain

The `data/mythic.onnx` can be retrained in the `train` step. It uses torchrun for multi-GPU parallelization.

```
torchrun --standalone --nproc-per-node=auto scripts/common/convert_model.py steps=train
```

This step will take a very long time, depending on your GPU configuration.

Output: The retrained ONNX `data/trained.onnx`

6.3.1 Hyper-parameters

Hyper-parameters (batch size, learning rate, ... etc) can be specified in a configuration file in `configs/yolopx/training_config/`. Two configs are provided:

Task	Training configuration file
360p training	config_360p.yaml
720p training	config_720p.yaml

To change parameter values, edit this file or create a new one in the same directory and put its name into the `TRAINING_SETUP` environment variable.

6.4 Optimize per-channel ADC ranges (optional)

This step currently does not improve accuracy.

```
python scripts/common/convert_model.py steps=optimize_ifsr
```

- **Output:** An optimized retrained ONNX `data/optimized_ifsr.onnx`

This ONNX can be used in the accuracy evaluation steps by using `trained_model=data/optimized_ifsr.onnx`.

MYTHIC

6.5 Compute Accuracy Metrics

6.5.1 M2000

Compute detailed metrics on the evaluation model with the command:

```
python3 scripts/common/convert_model.py steps=eval_trained
```

- **Output:** The accuracy metrics `data/metrics.json`.

Compute Monte Carlo-based 100PPM estimates:

```
python3 scripts/common/convert_model.py steps=mc_eval_trained  
mc_eval_trained.num_samples=100
```

- **Output:** One file per sample in `data/collected_metrics/`

Print the 100PPM estimates after collecting samples:

```
python3 scripts/common/convert_model.py steps=mc_summarize_metrics
```

- **Output:** The accuracy 100PPM metrics `data/collected_metrics/metrics_summary.json`

7 Convert to ACM Model

Convert the retrained `data/trained.onnx` to ACM model:

```
python scripts/common/convert_model.py steps=to_acm
```

- **Output:** The ACM model `data/acm.onnx`

7.1 Generate Artifact

Create an **artifact**, namely a set of files that includes the best model weights. The compiler will use the artifact to generate the firmware that is programmed into the Mythic AMP™ for inference.

To generate an artifact from the existing `data/acm.onnx` :

```
python3 scripts/common/convert_model.py steps=create_artifact
```

- **Output:** The training artifact `data/compiler_ready_artifact.tar.gz`

Alternatively, you can override the defaults with the following parameters:

Configuration parameter or group	Description	Default value
src	An ONNX evaluation model to generate an artifact from.	<code>\${acm_model}</code> , which defaults to <code>data/acm.onnx</code> .
dest	An ONNX file to save the converted model to. A value can be a local file name or a URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .	<code>\${env_setup.artifact_path}</code> , which defaults to <code>\${artifact}</code> , which default to <code>data/compiler_ready_artifact.tar.gz</code>

To specify a parameter value on the command line, use `create_artifact.parameter=value` syntax, e.g., `create_artifact.dest=gs://some_location/my_artifact.tar.gz` .

7.2 9. Firmware Compilation

Once the artifact has been generated, the output can be run with the compiler to create a binary that can be loaded for inference. To generate a firmware binary with the compiler, use the `mythic-compiler` script with a compiler configuration file located in `$MYTHIC_SDK_ROOT/configs/yolopx/compiler` .

```
mythic-compiler --compiler-config configs/yolopx/compiler/yolopx_720p.yaml \
  --input-artifact TRAINING_ARTIFACT.tar.gz --output-artifact FIRMWARE.tar.gz
```

Output: The firmware `FIRMWARE.tar.gz`

`mythic-compiler` takes the following parameters:

Parameter	Description
<code>--compiler-config</code>	A compiler recipe to use. Currently for the Pose model Mythic provides recipes for 640x640 and HxW=1280x720 image resolutions: <code>configs/yolov8/compiler/yolov8s_pose_640x640x3.yaml</code> . <code>configs/yolov8/compiler/yolov8s_pose_736x1280x3.yaml</code> .

MYTHIC

Parameter	Description
<code>-- input- artifac t</code>	A name of a training artifact generated by the steps described above.
<code>-- output- artifac t</code>	A name of a firmware artifact to be generated. A value can be a local file name or an URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .